SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1737

COURSE OUTCOME COVERED

CO1: *Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration:** 1 Hour

Total Marks: 30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I **Saai Vardhan D (192525205)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Saai Vardhan D

Question 1 – Hospital Shift Scheduling (Backtracking CSP) [10 marks]

1. Problem understanding – as a Constraint Satisfaction Problem

Component	Definition
Variables	D1, D2, D3 (the shift of each doctor)
Domain	{Morning, Afternoon, Night} for every variable
Constraints	(i) D1 ≠ Night (ii) D2 works before D3 (iii) one doctor per shift (all different) (iv) D3 ≠ Morning (v) every doctor gets exactly one shift

2. Method – Backtracking Search

Backtracking is depth-first search that assigns **one variable at a time** and checks the constraints after every assignment. If a constraint is violated the value is rejected and the next value is tried; if no value works, the search backtracks to the previous variable. Variable order: D1, D2, D3. Value order: Morning, Afternoon, Night.

3. Intermediate steps and constraint checks

Step	Assignment tried	Constraint checks	Result
1	D1 = Morning	(i) not Night ✓ (iii) Morning is free ✓	Accept
2	D2 = Morning	(iii) Morning already used by D1 ✗	Reject, try next value
3	D2 = Afternoon	(iii) Afternoon is free ✓ (ii) D2 can still precede D3, since Night is free ✓	Accept
4	D3 = Morning	(iii) Morning already used ✗ (iv) D3 cannot work Morning ✗	Reject, try next value
5	D3 = Afternoon	(iii) Afternoon already used by D2 ✗	Reject, try next value
6	D3 = Night	(iii) ✓ (iv) not Morning ✓ (ii) Afternoon < Night ✓ (v) all 3 doctors assigned ✓	**Accept – GOAL**

4. Final valid schedule

Morning	Afternoon	Night
D1	**D2**	**D3**

Verification: D1 is not on Night (i); D2 (Afternoon) works before D3 (Night) (ii); each shift has exactly one doctor (iii); D3 is not on Morning (iv); all three doctors are assigned exactly one shift (v). Continuing the search would give only one other solution (D1 = Afternoon, D2 = Morning, D3 = Night), so a valid schedule exists and is found with just 6 assignment attempts.

Question 2 – Robot Path Finding on a 5×5 Grid [10 marks]

1. Problem understanding

Grid cells are written as (row, column), counting from 0 at the top-left. **Start S = (0,0), Goal G = (4,4).** Obstacles (assumed, since the grid was not given): (0,1), (1,3), (2,0), (2,1), (3,3), (4,3). Moves: Up, Down, Left, Right, each costing 1; obstacles cannot be entered. Heuristic: Manhattan distance $h(n) = |r - 4| + |c - 4|$, so $h(S) = 8$.

2. Method – BFS (FIFO queue), then the Manhattan heuristic with a priority queue

All moves cost 1, so **BFS** finds the optimal path. It expands nodes in FIFO order (neighbours in the order Up, Down, Left, Right), skips obstacles and visited cells, and goal-tests when a node is generated. The same problem is then solved with a priority queue ordered by $f(n) = g(n) + h(n)$ to show how the heuristic reduces the work.

3. BFS – node expansion and queue updates

Step	Expand	New nodes added	Queue after expansion
1	(0,0)	(1,0)	(1,0)
2	(1,0)	(1,1)	(1,1)
3	(1,1)	(1,2)	(1,2)
4	(1,2)	(0,2), (2,2)	(0,2), (2,2)
5	(0,2)	(0,3)	(2,2), (0,3)
6	(2,2)	(3,2), (2,3)	(0,3), (3,2), (2,3)
7	(0,3)	(0,4)	(3,2), (2,3), (0,4)
8	(3,2)	(4,2), (3,1)	(2,3), (0,4), (4,2), (3,1)
9	(2,3)	(2,4)	(0,4), (4,2), (3,1), (2,4)
10	(0,4)	(1,4)	(4,2), (3,1), (2,4), (1,4)
11	(4,2)	(4,1)	(3,1), (2,4), (1,4), (4,1)
12	(3,1)	(3,0)	(2,4), (1,4), (4,1), (3,0)
13	(2,4)	(3,4)	(1,4), (4,1), (3,0), (3,4)
14	(1,4)	–	(4,1), (3,0), (3,4)
15	(4,1)	(4,0)	(3,0), (3,4), (4,0)
16	(3,0)	–	(3,4), (4,0)
17	(3,4)	(4,4)	(4,0), (4,4)

4. Priority-queue version (f = g + h)

Step	Node expanded	Nodes added	Priority queue (node : f)
1	(0,0) g=0, h=8, f=8	(1,0)	(1,0):8
2	(1,0) g=1, h=7, f=8	(1,1)	(1,1):8
3	(1,1) g=2, h=6, f=8	(1,2)	(1,2):8
4	(1,2) g=3, h=5, f=8	(0,2), (2,2)	(2,2):8, (0,2):10
5	(2,2) g=4, h=4, f=8	(3,2), (2,3)	(2,3):8, (3,2):8, (0,2):10
6	(2,3) g=5, h=3, f=8	(2,4)	(3,2):8, (2,4):8, (0,2):10
7	(3,2) g=5, h=3, f=8	(4,2), (3,1)	(2,4):8, (4,2):8, (0,2):10, (3,1):10
8	(2,4) g=6, h=2, f=8	(1,4), (3,4)	(4,2):8, (3,4):8, (0,2):10, (3,1):10, (1,4):10
9	(4,2) g=6, h=2, f=8	(4,1)	(3,4):8, (0,2):10, (3,1):10, (1,4):10, (4,1):10
10	(3,4) g=7, h=1, f=8	(4,4)	(4,4):8, (0,2):10, (3,1):10, (1,4):10, (4,1):10
11	(4,4) g=8, h=0, f=8	–	goal reached

5. Result and interpretation

Optimal path: (0,0) → (1,0) → (1,1) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → (4,4). **Cost = 8 moves.** This equals $h(S) = 8$, the Manhattan lower bound, so no cheaper path exists. BFS expanded 17 nodes, while the heuristic version needed only 11 queue removals because it explored only nodes with $f = 8$ first.

Question 3 – Autonomous Rescue Robot (Online Search) [10 marks]

a) How the constraints affect decision-making

Constraint	Effect on the robot's decisions
Moves: up, down, left, right	Only 4 actions per cell, no diagonal shortcuts; every path is a sequence of grid steps.
Blocked cells	Removed from the search space; a planned path can become invalid, so the robot must be able to replan.

Constraint	Effect on the robot's decisions
Sensing adjacent cells only	Partially observable: the robot cannot plan with certainty, so it keeps a knowledge map and treats unseen cells as free until sensed (optimism).
Cost 1 per move, +2 for a risky zone	Step costs differ (1 or 3), so BFS is not suitable; a cost-sensitive search is needed. A longer but safer route may be cheaper than a short risky one.
Minimise cost, ensure safety	Objective = least total cost. Risky cells are penalised, not forbidden, and are used only when the detour costs more.
Dynamic knowledge update	Planning and acting are interleaved (online search): sense, update, replan, move.

Priority: hard constraints first (moves, blocked cells), then the cost objective (risk penalty).

b) Solution approach – Online agent using Uniform Cost Search with replanning

1. Initialise: knowledge map holds S and G; unknown cells are assumed free (cost 1).

2. Sense and update: observe the four adjacent cells and record blocked or risky ones.

3. Plan: run UCS on the known map, always expanding the lowest path-cost node (step cost 1, or 3 for a risky cell).

4. Act: execute only the first move of the plan, then sense again.

5. Replan: if the new percept blocks the path or reveals a risky cell, rerun UCS from the current cell. Repeat until G is reached; if no path exists on the known map, explore the nearest unknown cell.

Demonstration: the initial plan costs 8, but the robot discovers (2,0) blocked and later (1,2), (2,2), (3,2) blocked and (4,1) risky. At (3,1) the detour over the top row would cost at least 10, while the risky route costs 6, so it takes the risky cell. Executed path: (0,0) → (1,0) → (1,1) → (2,1) → (3,1) → (4,1) → (4,2) → (4,3) → (4,4); **actual cost = 10**.

c) Application of AI concepts

Concept	Application
Intelligent agent	Percepts: status of adjacent cells and own position. Actions: Up, Down, Left, Right. Performance: minimum total cost while reaching the survivor safely. A model-based, goal-based and utility-based agent, since it keeps a map and compares path costs.
Rationality	At each step it picks the move on the least-cost path given what it knows, and replans when new percepts arrive. It is rational, not omniscient: the plan cost of 8 rose to 10 only because new information appeared.
Environment type	Partially observable, single agent, sequential, discrete, initially unknown, and essentially static during navigation (dynamic if rubble shifts). Moves are deterministic.

d) Why this approach is most suitable

Offline UCS or A* needs the full map, which the robot does not have; online replanning copes with partial observability.

BFS and DFS ignore step costs, so they cannot weigh a risky shortcut against a safe detour; greedy search can get trapped behind blocked cells.

UCS is complete and optimal when all costs are positive, and on a small grid replanning at every step is cheap. Safety is built into the cost, giving a safe, minimum-cost route on current knowledge.